# Universidad Tecnológica del Norte de Guanajuato

## Ingeniería en Redes Inteligentes y Ciberseguridad

### Automatización de Infraestructura Digital
### Unidad I - Entornos de desarrollo en la automatización de redes

**Modalidad:** Ordinario

**Alumno:** Méndez Fernández Alan Eusebio

**Grupo:** GIRI6091

**Número de control:** 1223100825

**Fecha:** 12 de octubre de 2026

Reporte basado en el proyecto proporcionado por el profesor.

## Contenido

1. Introducción
2. Objetivo
3. Herramientas utilizadas
4. Instalación y configuración del entorno
5. Configuración del sistema de ventas
6. Pruebas y capturas
7. Publicación en GitHub
8. Lista de verificación y resultados
9. Conclusión
10. Anexo: recursos de la comunidad de colaboración
11. Referencias bibliográficas

## 1. Introducción

En esta actividad trabajé con Docker para poner en funcionamiento un sistema de ventas que nos proporcionó el profesor. La idea era preparar un entorno donde pudiera ejecutar varias partes de una aplicación sin tener que instalar cada una directamente en mi equipo. Antes de comenzar, ya tenía algunas herramientas instaladas en Linux, así que primero revisé sus versiones y confirmé que estuvieran funcionando. Después descargué el repositorio de la práctica y preparé los archivos que necesitaba para levantar la aplicación.

El sistema está dividido en cuatro servicios. Por un lado está el frontend, que es la página con la que interactúa el usuario. También tiene un backend desarrollado con Node.js, encargado de atender las peticiones y consultar la información. Para guardar los datos utiliza MySQL y para revisar la base de datos se usa phpMyAdmin. Todos estos servicios tenían que trabajar juntos, respetando los nombres, puertos y versiones indicados en la rúbrica.

Durante el desarrollo utilicé Docker Desktop y Docker Compose, además de Git y Visual Studio Code. Con Docker Compose pude colocar la configuración en archivos YAML y manejar los contenedores del mismo proyecto. Git me sirvió para organizar los cambios y publicar mi trabajo en GitHub, conservando el enlace al repositorio original del profesor.

No todo funcionó desde el primer intento. Aunque las imágenes se construyeron, el backend presentó un error al conectarse con MySQL. Revisé los mensajes de error, identifiqué el problema de autenticación y apliqué un ajuste al usuario de la base de datos de esta práctica. Después pude comprobar que los cuatro servicios estaban activos y entrar al sistema de ventas. En este README explico los pasos que seguí, los comandos que utilicé, las pruebas que realicé y las capturas que guardé como parte del trabajo.

## 2. Objetivo

Mi objetivo en esta práctica fue preparar y comprobar un entorno de trabajo con Docker y Docker Compose. Para hacerlo, utilicé el sistema de ventas que compartió el profesor y fui revisando que el frontend, el backend, MySQL y phpMyAdmin funcionaran juntos en mi equipo con Linux.

## 3. Herramientas utilizadas

### 3.1 Docker Engine

Para esta práctica usé Docker porque me permite ejecutar cada parte de la aplicación en su propio contenedor. Así no tuve que instalar MySQL ni Node.js directamente para poner a funcionar el proyecto. Mientras trabajaba entendí mejor la diferencia entre una **imagen**, que sirve como base, y un **contenedor**, que es lo que Docker crea y ejecuta a partir de esa imagen. También aprendí a usar un **volumen** para que MySQL conserve sus datos al detener los contenedores.

### 3.2 Docker Compose

Con Docker Compose reuní la configuración de los cuatro servicios en archivos `.yml`. Me resultó más sencillo que iniciarlos uno por uno, porque en el mismo archivo pude revisar los puertos, nombres, imágenes, variables, volúmenes y dependencias. Dejé esa configuración en `stack-iniciales-alumno.yml` y `stack-aemf.yml`. Para construir e iniciar la aplicación utilicé `docker compose up -d --build`.

### 3.3 Docker y Swagger

En la rúbrica también encontré el término “Docker Swagger”, por lo que revisé para qué sirve Swagger. Entendí que permite consultar y documentar una API, por ejemplo con Swagger UI y OpenAPI, y que también puede ejecutarse dentro de un contenedor. Como el repositorio que usamos no incluye una especificación OpenAPI ni pide configurar ese servicio aparte, decidí describir la herramienta sin agregar otro contenedor al sistema de ventas.

### 3.4 Visual Studio Code y extensiones

Usé Visual Studio Code para revisar los Dockerfile, los archivos YAML, la configuración de Nginx y el README. Con las extensiones de Docker y Containers pude identificar mejor los archivos relacionados con los contenedores. La extensión YAML me ayudó a leer la configuración y con GitLens pude revisar los cambios. También comprobé desde la terminal que estas extensiones estuvieran instaladas.

En VS Code también abrí el archivo `stack-iniciales-alumno.yml`, donde se pueden ver los servicios y los datos de configuración de MySQL y phpMyAdmin.




Archivo YAML abierto en VS Code con los servicios del sistema de ventas

### 3.5 Git y GitHub

Con Git fui registrando los cambios del proyecto. GitHub me permitió guardar la entrega en un repositorio personal para que el profesor pueda revisar el código, los archivos de configuración y las evidencias. No subí los cambios al repositorio original: lo dejé como referencia con el nombre `upstream` y usé `origin` para mi repositorio.

## 4. Instalación y configuración del entorno

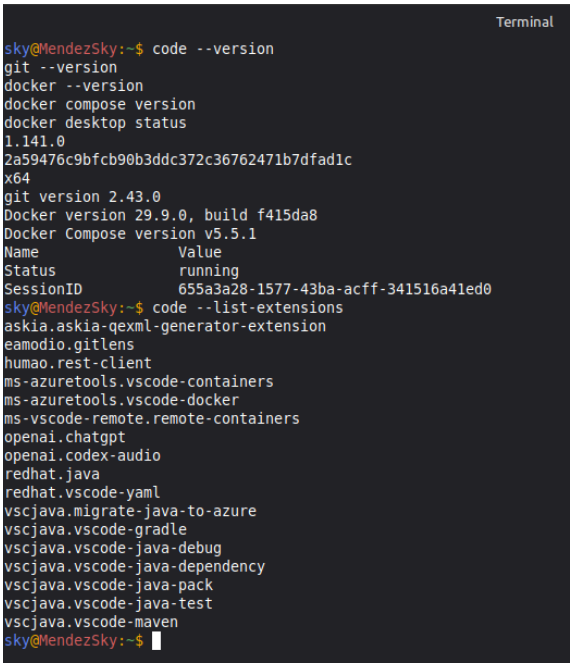
### 4.1 Revisión de las herramientas en Linux

Lo primero que hice fue comprobar las versiones de los programas con los que iba a trabajar. Como ya tenía Docker Desktop, Git y VS Code instalados, no fue necesario volver a instalarlos para realizar la práctica.

```
uname -a
cat /etc/os-release
code --version
git --version
docker --version
docker compose version
docker desktop status
code --list-extensions
```

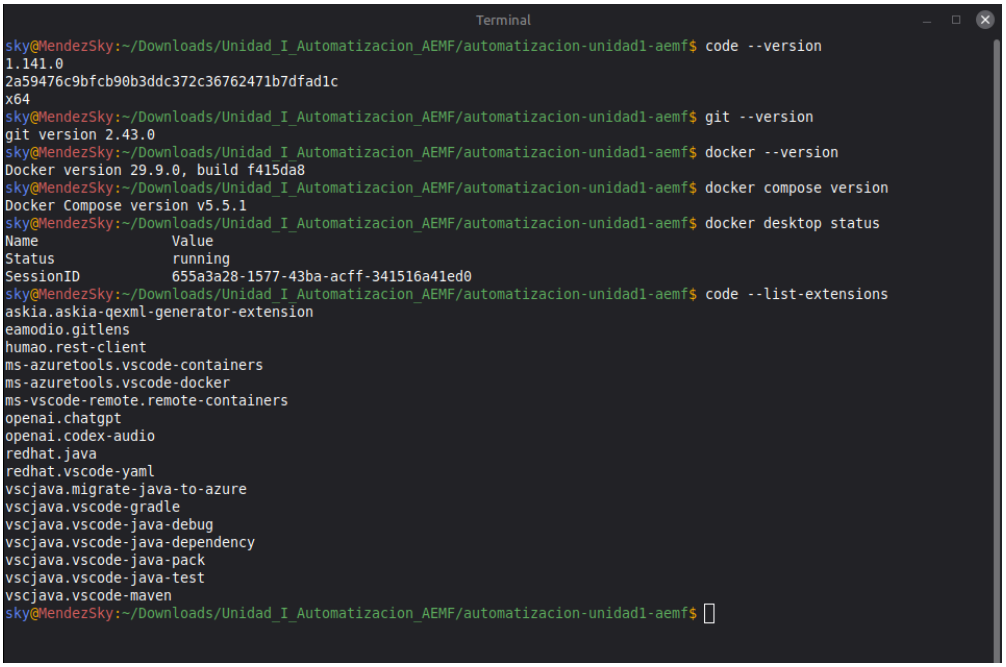
Estos fueron los datos que obtuve en la terminal:

| Herramienta | Versión o resultado |
|---|---|
| Visual Studio Code | 1.141.0 |
| Git | 2.43.0 |
| Docker (cliente) | 29.9.0 |
| Docker Compose | v5.5.1 |
| Docker Desktop | `running` |
| Extensiones | Docker, Containers, YAML y GitLens, entre otras |



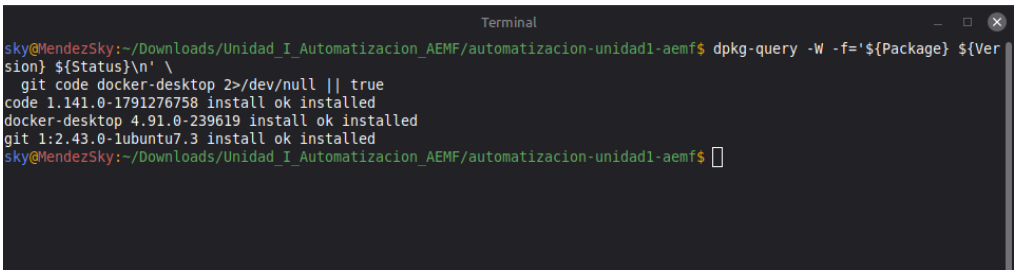

Versiones de las herramientas y extensiones instaladas

Volví a consultar las versiones, el estado de Docker Desktop y la lista de extensiones para dejar otra evidencia de las herramientas que utilicé.




Versiones de VS Code, Git, Docker y extensiones en mi terminal

Además, consulté los paquetes que Ubuntu tiene registrados como instalados. Se puede ver que Git, VS Code y Docker Desktop ya estaban disponibles en mi equipo; no tuve que reinstalarlos.




Paquetes Git, VS Code y Docker Desktop instalados según dpkg

## 4.2 Cómo se instalan las herramientas

Aunque yo ya tenía instaladas las herramientas, quise dejar escritos los comandos de instalación para que se entienda cómo prepararía otro equipo con Ubuntu o una distribución basada en Debian. En mi computadora no tuve que reinstalarlas: comprobé que ya estaban disponibles y funcionaban.

**Git.** Se puede instalar desde la terminal con:

```
sudo apt update
sudo apt install -y git
git --version
```

En mi caso, al ejecutar `sudo apt install git -y`, el sistema indicó que Git ya estaba instalado y que no había paquetes nuevos que agregar.

**Visual Studio Code.** Si necesitara instalarlo en otra computadora, descargaría el archivo `.deb` para Linux desde su página oficial y lo instalaría desde la carpeta donde lo guardé:

```
sudo apt install ./NOMBRE_DEL_ARCHIVO.deb
code --version
```

Para los complementos usados en esta actividad, los comandos son:

```
code --install-extension ms-azuretools.vscode-docker
code --install-extension ms-azuretools.vscode-containers
```

```
code --install-extension redhat.vscode-yaml
code --install-extension eamodio.gitlens
code --list-extensions
```

**Docker.** Si tuviera que preparar otro equipo con Ubuntu, primero agregaría el repositorio oficial de Docker y después instalaría los paquetes. Estos son los comandos que consulté para documentar ese procedimiento:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://
download.docker.com/linux/ubuntu $(. /etc/os-release && echo ${UBUNTU_CODENAME:-$VERSION_CODENAME})
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Para utilizar Docker Desktop en Linux, primero revisaría que KVM esté disponible. Después descargaría el paquete `.deb` desde la página oficial de Docker Desktop, indicaría la ruta donde lo guardé e iniciaría el servicio para seleccionar el contexto `desktop-linux`:

```
sudo apt install /ruta/al/archivo/docker-desktop-amd64.deb
systemctl --user start docker-desktop
docker context use desktop-linux
docker desktop status
```

**En mi equipo no ejecuté estas instalaciones desde cero**, porque Docker Desktop ya estaba instalado. Lo que sí comprobé fue que el contexto `desktop-linux` se conectara al servidor y que `hello-world` funcionara. Docker Compose venía disponible con Docker y lo revisé con `docker compose version`. Para una instalación nueva es importante seguir la versión más reciente de las instrucciones oficiales de Docker.

### 4.3 Descarga del proyecto del profesor

Tomé como base el repositorio indicado en el instrumento. El comando para descargarlo es:

```
git clone https://github.com/edomenzain/automatizacion-redes.git
cd automatizacion-redes
```

Para la entrega trabajé sobre la revisión `4e50b4c66092384d27a92265e29a81fbda44205d`, con una rama llamada `entrega-unidad1-aemf`. También guardé el README original del docente en `docs/README_PROFESOR_ORIGINAL.MD` para conservar las instrucciones con las que comencé.

## 5. Configuración del sistema de ventas

### 5.1 Los cuatro servicios

En el archivo `stack-iniciales-alumno.yml` coloqué los servicios solicitados. También dejé `stack-aemf.yml` con la misma configuración para trabajar usando mis iniciales.

| Servicio | Imagen utilizada | Nombre del contenedor | Puerto de mi equipo |
|---|---|---|---|
| MySQL | `mysql:8.0` | `mysql-db` | `3306` |
| phpMyAdmin | `phpmyadmin:latest` | `phpmyadmin` | `9090` |
| Backend | `node:18.17.1-alpine` | `sistema-ventas-back` | `8080` |
| Frontend | `nginx:stable-alpine` | `sistema-ventas-front` | `80` |

Para MySQL utilicé la base `sistema_ventas_db`, el usuario `admin` y las contraseñas de prueba indicadas por el profesor. Su información se guarda en el volumen `dbfiles`. Además, el SQL `dbfiles/sistema_ventas_db_rev1.1.sql` contiene las tablas y los datos iniciales de la aplicación.

### 5.2 Dockerfile del backend

Preparé `sistema-ventas-back/Dockerfile` con la imagen `node:18.17.1-alpine`, como pide el profesor. Ahí indiqué que se trabajara en `/usr/src/app`, que se copiaran `package.json` y `package-lock.json`, que se ejecutara `npm install` y que después se copiara el resto del backend. También dejé expuesto el puerto `3000` y el comando `node index.js`. Desde el navegador pude entrar por `localhost:8080`, porque Compose conecta ese puerto de mi equipo con el `3000` del contenedor.

Este es el contenido del Dockerfile que utilicé:

```
FROM node:18.17.1-alpine
WORKDIR /usr/src/app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

### 5.3 Dockerfile del frontend

Para el frontend utilicé `nginx:stable-alpine`. En su Dockerfile declaré el volumen `/temp`, eliminé los archivos que trae Nginx en su carpeta web y copié `nginx.conf`, `mime.types` y los archivos compilados de `sistema-ventas-front`. Finalmente, dejé expuesto el puerto `80` y el comando `nginx -g 'daemon off;'` para mantener el servidor funcionando.

En este caso el Dockerfile del frontend quedó así:

```
FROM nginx:stable-alpine
VOLUME /temp
RUN rm -rf /usr/share/nginx/html/*
COPY nginx.conf /etc/nginx/nginx.conf
COPY mime.types /etc/nginx/mime.types
COPY sistema-ventas-front/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

### 5.4 Levantar los contenedores

Antes de ejecutar los servicios revisé que estuvieran libres los puertos 80, 3306, 8080 y 9090. Después validé el YAML, construí las imágenes y comprobé el estado de los contenedores.

```
sudo ss -ltnp | grep -E ':(80|3306|8080|9090)\b' || true

docker compose -f stack-aemf.yml config -q
docker compose -f stack-aemf.yml up -d --build
docker compose -f stack-aemf.yml ps -a
docker compose -f stack-aemf.yml logs --tail 100
```

La aplicación quedó organizada de esta manera:

```
Navegador
  |-- localhost:80 -----> Frontend Nginx (Angular)
  |-- localhost:8080 ---> Backend Node.js (puerto 3000)
  |                          |
  |                          v
  |                       MySQL (mysql-db:3306)
  |                          |
  |                       Volumen dbfiles
  |
  `-- localhost:9090 ---> phpMyAdmin ---> MySQL
```

Para detener solamente los contenedores de esta práctica puedo utilizar `docker compose -f stack-aemf.yml down`. No usé `down -v`, porque esa opción también borraría el volumen donde se guardan los datos de MySQL.

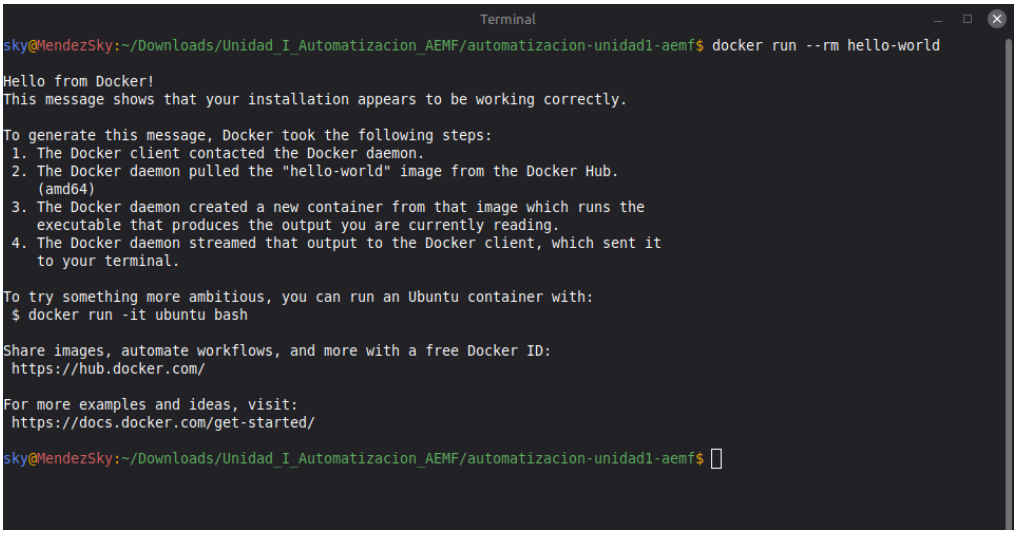
## 6. Pruebas y capturas

### 6.1 Primera prueba: hello-world

Empecé con la prueba más básica de Docker:

```
docker run --rm hello-world
```

En la terminal apareció `Hello from Docker!`, lo que confirmó que Docker podía descargar una imagen y ejecutar un contenedor.




Prueba hello-world en la terminal

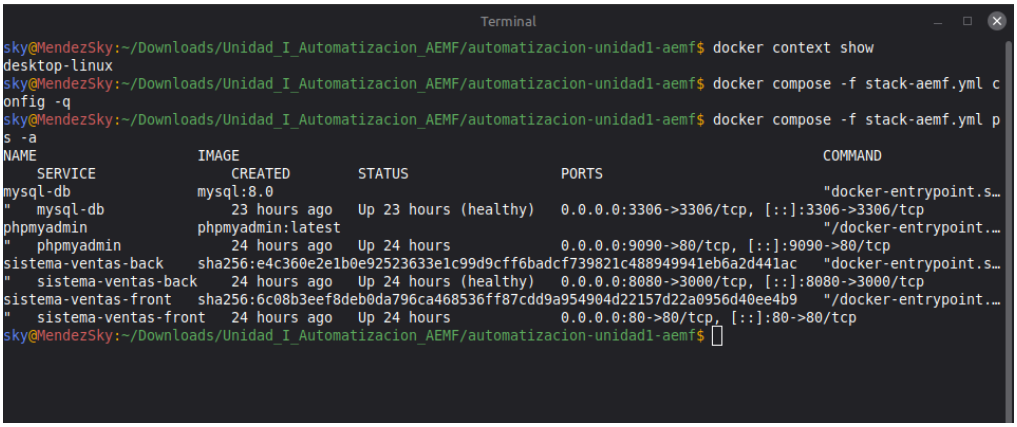
## 6.2 Revisión de Docker Compose

Para ver tanto el archivo de configuración como los contenedores, utilicé:

```
docker compose -f stack-aemf.yml config --services
docker compose -f stack-aemf.yml config -q
docker compose -f stack-aemf.yml ps -a
```

Después validé el archivo YAML y levanté la aplicación con los comandos indicados en el apartado 5. Al revisar el resultado, MySQL y el backend aparecieron como `healthy`, mientras que el frontend y phpMyAdmin estaban en ejecución. De esta forma comprobé que los cuatro servicios estaban activos.

En esta captura aparece la salida de `docker compose -f stack-aemf.yml ps -a`: MySQL y el backend están saludables, mientras que phpMyAdmin y el frontend permanecen en ejecución. También validé el YAML con `config -q`, que no mostró errores.



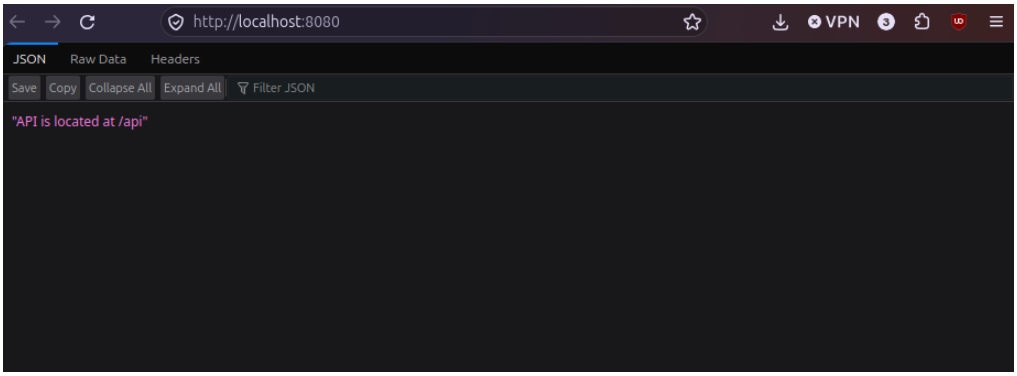

Cuatro servicios ejecutándose en Docker Compose y validación del YAML

## 6.3 Respuesta del backend

Para revisar el backend ejecuté:

```
curl -i http://localhost:8080/
```

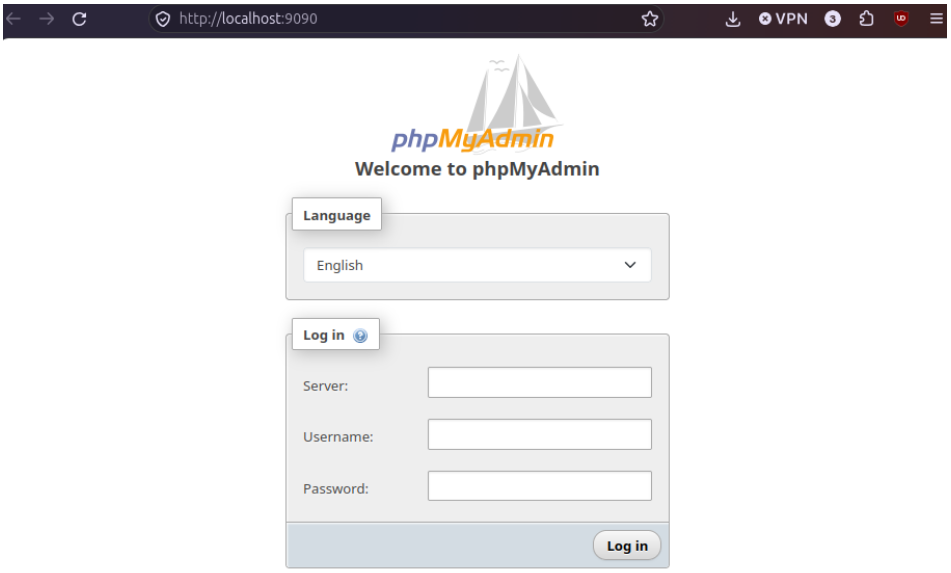
Obtuve una respuesta HTTP `200 OK` y el mensaje `"API is located at /api"`. También abrí esa dirección en el navegador.



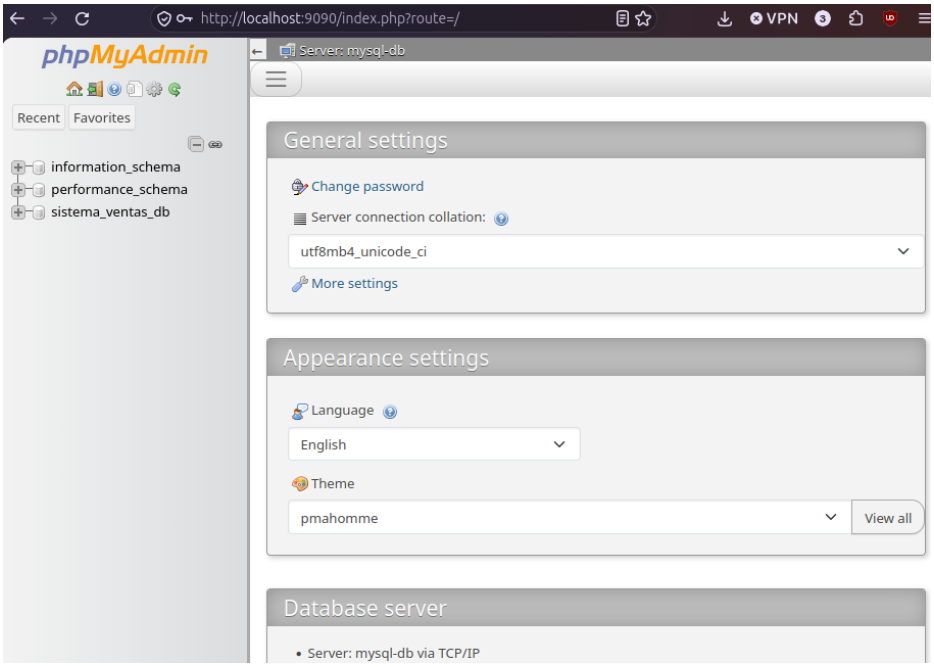

Respuesta del backend en el navegador

## 6.4 Base de datos y phpMyAdmin

Entré a phpMyAdmin desde `http://localhost:9090/` usando el servidor `mysql-db`. Ahí pude ver la base de datos `sistema_ventas_db` creada con la información del proyecto.




Pantalla de acceso a phpMyAdmin




Base de datos del sistema de ventas en phpMyAdmin

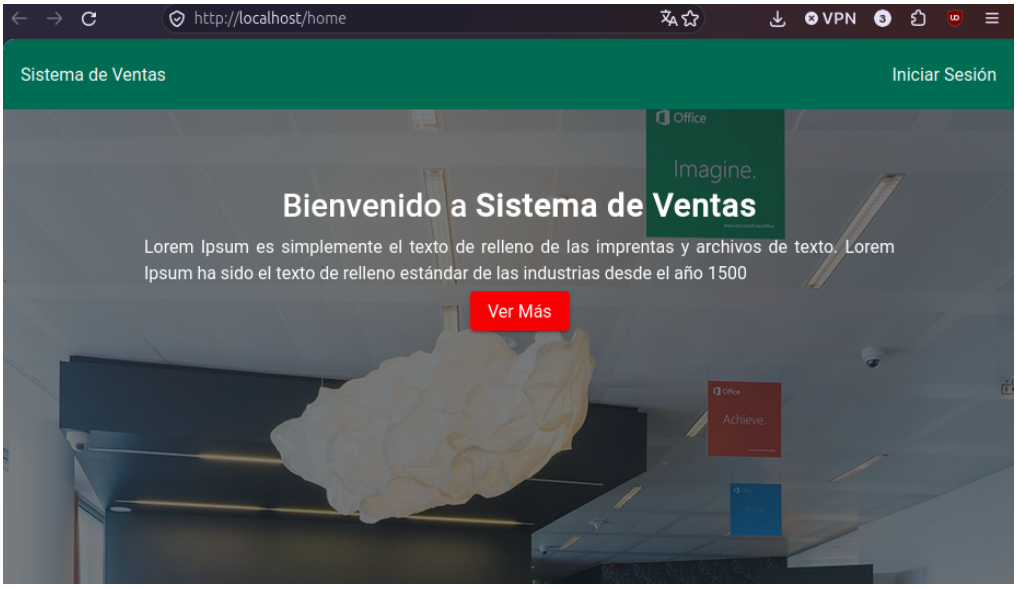
También utilicé este comando para consultar las tablas y revisar el usuario cargado por el SQL:

```
docker exec mysql-db sh -c 'MYSQL_PWD=admin mysql -uadmin sistema_ventas_db -e "SHOW TABLES; SELECT
username FROM tbl_usuario LIMIT 3;"'
```

Al revisar el archivo SQL del proyecto identifiqué estas tablas: `tbl_usuario`, `tbl_rol`, `tbl_rol_usuario`, `tbl_categoria`, `tbl_producto`, `tbl_venta`, `tbl_detalle_venta` y `tbl_imagen_producto`.

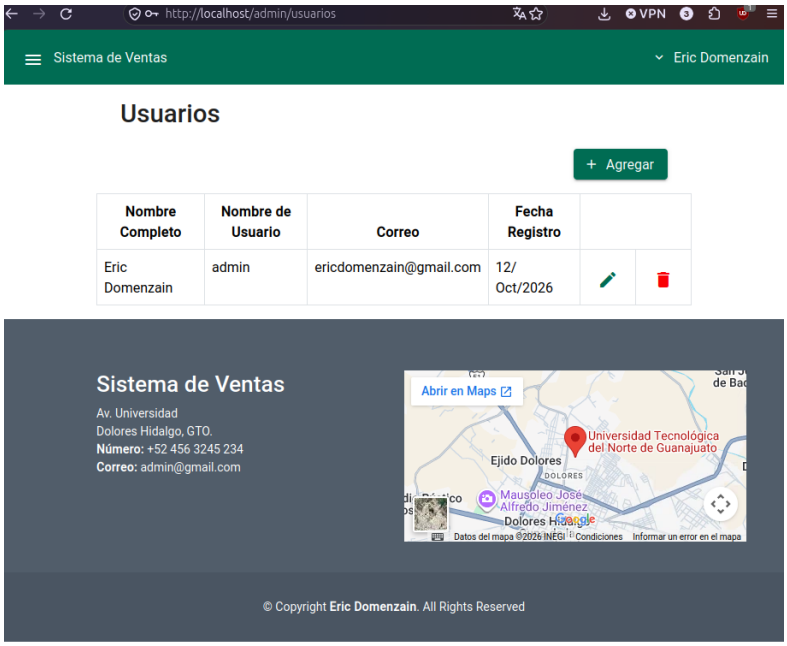
## 6.5 Página de inicio e ingreso como administrador

Abrí `http://localhost/home` y apareció la página principal del sistema de ventas.




Página principal del sistema de ventas

Luego inicié sesión con el usuario `admin` y la contraseña `12345678`, que son los datos de prueba proporcionados para esta actividad. El sistema me permitió acceder a la sección de usuarios del administrador.




Acceso al área administrativa y listado de usuarios

La autenticación también se puede consultar desde la API con:

```
curl -i -X POST http://localhost:8080/api/auth \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"12345678"}'
```

Cuando guardo el resultado de esta petición, oculto el token JWT antes de compartirlo. Para eso agregué al script de verificación una salida donde se conserva el resultado del inicio de sesión, pero no el token completo.

## 6.6 Error que encontré y cómo lo resolví

Al principio el backend no funcionaba bien. En Docker aparecía `unhealthy` y, cuando abría `localhost:8080`, respondía con un error `500`. MySQL sí estaba funcionando, por lo que revisé la conexión desde el contenedor de Node.js. Ahí apareció el error `ER_NOT_SUPPORTED_AUTH_MODE`.

El problema era que el usuario `admin` de MySQL estaba usando `caching_sha2_password` y el cliente de base de datos que trae el proyecto no era compatible con ese método. Para solucionarlo, cambié la autenticación de ese usuario a `mysql_native_password` dentro del MySQL de esta práctica. Después la página del backend respondió con HTTP `200` y los cuatro servicios pudieron iniciarse correctamente.

Dejé ese mismo ajuste en `dbfiles/02_mysql_native_auth.sql`, que está referenciado en los dos archivos Compose para las instalaciones nuevas. Primero comprobé la corrección sobre la base que ya tenía funcionando y después la repetí con contenedores temporales y una base recién inicializada, como explico en el apartado 6.8. `mysql_native_password` y las contraseñas del ejemplo se utilizan solo para este laboratorio, no como configuración recomendada para producción.

## 6.7 Cómo reuní las pruebas en un solo comando

Para no repetir manualmente cada comprobación, preparé `scripts/verificar.sh`. Este archivo ejecuta las pruebas de versiones, `hello-world`, validación del YAML, contenedores, backend, base de datos e inicio de sesión. Es un script de Bash que dejé dentro del proyecto para poder repetir las verificaciones cuando lo necesite.

```
./scripts/verificar.sh
```

El script guarda sus resultados en `evidencias/salidas/`. Para el reporte utilicé las capturas que tomé mientras trabajaba en mi equipo, y dejé los comandos para repetir las comprobaciones.

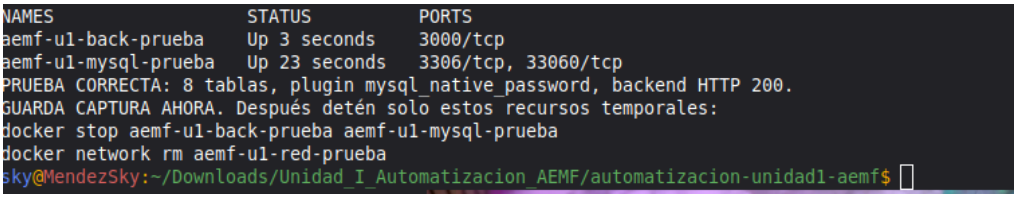
### 6.8 Prueba de MySQL con una instalación nueva

Para comprobar que el problema de autenticación no regresara al instalar la aplicación en otro equipo, hice una prueba adicional con una base MySQL nueva. Usé el script `scripts/probar_mysql_limpio.sh`, que creó dos contenedores temporales y una red propia, sin publicar puertos ni reutilizar el volumen de MySQL de mi aplicación principal.

Lo ejecuté desde la carpeta del proyecto:

```
mkdir -p evidencias/salidas
set -o pipefail
./scripts/probar_mysql_limpio.sh 2>&1 | tee evidencias/salidas/14-prueba-mysql-limpio.txt
```

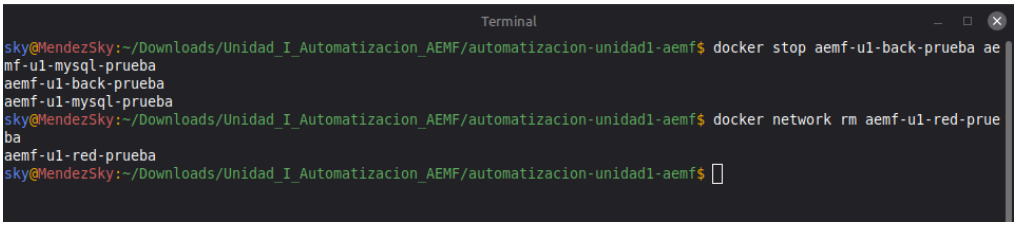
El resultado final fue **`PRUEBA CORRECTA: 8 tablas, plugin mysql_native_password, backend HTTP 200`**. El script revisó que las tablas se hubieran importado, que el usuario `admin` empleara el método de autenticación esperado y que el backend temporal respondiera correctamente.




Resultado de la prueba limpia de MySQL: ocho tablas, autenticación correcta y backend HTTP 200

Cuando terminé, detuve únicamente los dos contenedores de prueba y eliminé la red temporal:

```
docker stop aemf-u1-back-prueba aemf-u1-mysql-prueba
docker network rm aemf-u1-red-prueba
```




Comandos y resultado de la limpieza de los contenedores y la red temporales

Con esto pude probar el arranque inicial sin borrar `automatizacion-unidad1-aemf_dbfiles` ni modificar los contenedores de mis otros proyectos.

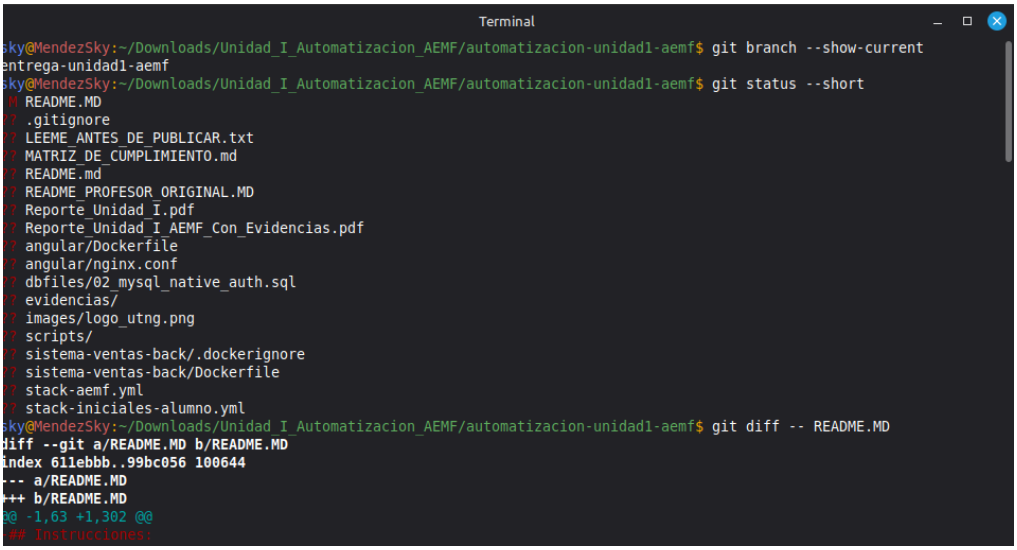
## 7. Publicación en GitHub

Para entregar mi práctica creé el repositorio automatizacion-infraestructura-digital-u1 en mi cuenta de GitHub. Estos fueron los pasos que seguí:

### 7.1 Preparé mi rama de trabajo

Descargué el repositorio del profesor y utilicé la rama `entrega-unidad1-aemf` para mantener organizados mis archivos. Revisé los cambios con:

```
git branch --show-current
git status --short
```




Rama de trabajo y archivos de la práctica

## 7.2 Separé el repositorio del profesor del mío

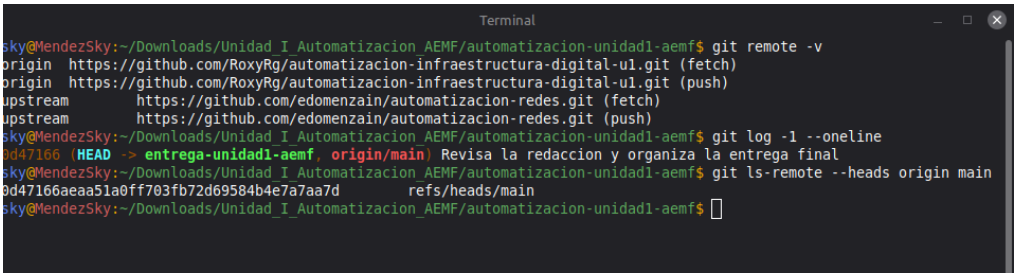
Para no enviar cambios al repositorio original, cambié el nombre de su remoto a `upstream`:

```
git remote rename origin upstream
git remote -v
```




Remoto del profesor conservado como upstream

Más adelante revisé de nuevo los remotos y el commit que tenía publicado en ese momento. En la terminal quedó registrada mi rama y el repositorio al que envié la práctica.




Revisión del remoto origin, upstream y el commit publicado

## 7.3 Creé y conecté mi repositorio

Primero creé en GitHub el repositorio público `automatizacion-infraestructura-digital-u1`. Elegí dejarlo vacío porque ya tenía preparado mi README. Después lo conecté con mi carpeta de trabajo mediante el remoto `origin`:

```
git remote add origin https://github.com/RoxyRg/automatizacion-infraestructura-digital-u1.git
git remote -v
```

## 7.4 Guardé los cambios y los publiqué

Revisé qué archivos iba a subir, preparé los del proyecto con `git add` y guardé el primer commit. Después publiqué la rama en GitHub.

```
git add README.md angular/Dockerfile angular/nginx.conf \
  sistema-ventas-back/Dockerfile stack-aemf.yml \
  stack-iniciales-alumno.yml dbfiles/02_mysql_native_auth.sql

git diff --cached --stat
git commit -m "Entrega Unidad I - Automatizacion de Infraestructura Digital"
git push -u origin HEAD:main
```

El primer commit fue `89a6a04`. Después seguí corrigiendo el README y el PDF; también quedó publicado el commit `0d47166`, en el que organicé la entrega. El historial de estos cambios se puede consultar en mi repositorio. También guardé una comprobación de la rama remota en `evidencias/salidas/13-git-publicacion.txt`.

Finalmente abrí el repositorio desde el navegador y comprobé que GitHub mostraba el README con el logotipo de la UTNG y los datos de mi práctica.




README de la práctica publicado en GitHub

## 8. Lista de verificación y resultados

Para revisar que el entorno funcionara, fui comprobando los servicios por separado. En esta tabla anoté las pruebas que sí realicé y dónde dejé sus evidencias:

| Qué revisé | Resultado que comprobé | Evidencia que guardé |
|---|---|---|
| Docker | Ejecuté `hello-world` y apareció el mensaje de éxito. | Captura 01. |
| Docker Compose | Validé el YAML y vi los cuatro servicios levantados. | Captura 10, con estado de todos los servicios. |
| MySQL | El contenedor mostró estado `healthy` y la base `sistema_ventas_db` estaba disponible. | Captura 05 y salida del servicio. |
| phpMyAdmin | Pude iniciar sesión y consultar la base del ejercicio. | Capturas 04 y 05. |
| Backend | La dirección `localhost:8080` respondió con HTTP `200`. | Captura 03. |
| Frontend | Abrí la página principal y entré a la pantalla administrativa. | Capturas 02 y 06. |
| Herramientas | Revisé las versiones, las extensiones y los paquetes instalados. | Capturas 07, 13 y 13b. |

| Qué revisé | Resultado que comprobé | Evidencia que guardé |
|---|---|---|
| GitHub | Publiqué mi README, archivos de Docker, PDF y capturas en mi repositorio. | Capturas 08, 09, 15 y 16; registro `13-git-publicacion.txt`. |

MySQL nuevo y conexión del backend | Probé la inicialización con ocho tablas, `mysql_native_password` y HTTP 200 desde un backend temporal. | Capturas 14 y 14b; apartado 6.8. |

Además, conservé los comandos del apartado 6 para repetir las pruebas si necesito volver a levantar la aplicación.

## 9. Conclusión

**Méndez Fernández Alan Eusebio**

En esta práctica aprendí que trabajar con Docker no es solamente descargar imágenes y prender contenedores. Para que una aplicación funcione se necesita revisar cómo se conectan sus servicios, qué puertos utiliza cada uno y de dónde obtiene los datos. Al principio pensé que, si Docker lograba construir las imágenes, ya tendría casi todo listo. Sin embargo, cuando intenté abrir el backend me encontré con un error y tuve que revisar con más detalle la configuración.

Lo que más trabajo me dio fue la conexión entre Node.js y MySQL. El contenedor de MySQL aparecía saludable, pero el backend seguía respondiendo con un error 500. Primero revisé los registros y después hice una prueba directa de conexión desde el contenedor. Así encontré el mensaje `ER_NOT_SUPPORTED_AUTH_MODE` y pude identificar que se trataba del método de autenticación. Una vez que ajusté el usuario de la base de datos para esta práctica, el backend comenzó a responder correctamente. Eso me ayudó a entender que un contenedor puede estar encendido y aun así tener problemas para realizar su función.

También me sirvió aprender a usar Docker Compose, porque tener todos los servicios descritos en un mismo archivo facilita revisar sus configuraciones y volver a iniciarlos cuando sea necesario. Me quedó más claro para qué sirven los volúmenes, especialmente en MySQL, donde no quiero perder las tablas ni los usuarios cada vez que detengo un contenedor. Además, pude comprobar la diferencia entre acceder a un servicio desde el navegador mediante `localhost` y comunicar contenedores usando nombres internos como `mysql-db`.

Otra parte importante fue trabajar con Git. Antes de publicar, revisé los archivos que había modificado y separé el repositorio del profesor de mi propio repositorio para no confundir los cambios. Después hice el commit y comprobé que apareciera en GitHub. Al finalizar, tenía el sistema de ventas funcionando, sus configuraciones guardadas y las capturas de las pruebas. También pude comprobar con una base nueva que el ajuste de MySQL se aplicaba sin repetirlo manualmente. Me llevo de esta actividad la importancia de revisar los errores con calma, entender qué está fallando y dejar escrito el procedimiento para poder repetirlo más adelante.

## 10. Anexo: recursos de la comunidad de colaboración

Como apoyo para resolver dudas técnicas, consulté el repositorio del profesor y recursos de la comunidad que explican cómo funcionan las herramientas de esta actividad:

| Recurso | Para qué lo utilicé |
|---|---|
| Repositorio del profesor | Código del sistema de ventas y script SQL. |
| Docker Docs | Comandos y funcionamiento de contenedores. |
| Docker Compose | Configuración de servicios en YAML. |
| Docker Hub: MySQL | Imagen de MySQL y opciones de inicialización. |
| Docker Hub: phpMyAdmin | Imagen de phpMyAdmin. |
| Documentación de Node.js | Funcionamiento del backend. |
| Documentación de Nginx | Configuración del frontend. |
| Swagger | Conceptos de Swagger y OpenAPI. |
| VS Code | Editor y extensiones. |
| Git | Comandos para registrar y publicar los cambios. |

## 11. Referencias bibliográficas (APA 7)


Docker, Inc. (s. f.). *Docker documentation.* https://docs.docker.com/

Docker, Inc. (s. f.). *Docker Compose overview.* https://docs.docker.com/compose/

Docker, Inc. (s. f.). *MySQL - Official image.* Docker Hub. https://hub.docker.com/_/mysql

Docker, Inc. (s. f.). *phpMyAdmin - Official image.* Docker Hub. https://hub.docker.com/_/phpmyadmin

Domenzain, E. (s. f.). *Automatización redes* [Repositorio de código]. GitHub. https://github.com/edomenzain/automatizacion-redes

Git. (s. f.). *Git documentation.* https://git-scm.com/doc

NGINX. (s. f.). *NGINX documentation.* https://nginx.org/en/docs/

OpenAPI Initiative. (s. f.). *OpenAPI specification.* https://spec.openapis.org/oas/latest.html

Visual Studio Code. (s. f.). *Documentation.* Microsoft. https://code.visualstudio.com/docs